

Spécification et Validations de Circuits Électroniques

Projet d'Ingénierie Dirigée par les Modèles

L'électronique embarquée connaît depuis quelques années un essor sans précédent, dû à la miniaturisation des composants mais aussi et surtout à la grande fiabilité des processus de fabrication, permettant de pousser toujours plus loin la complexité des designs.

Aujourd'hui, nous nous intéressons à ce que l'on pourrait qualifier d'embarqué *extrême*, c'est-à-dire de systèmes qui, une fois déployés, ne peuvent plus être maintenus que par des passerelles rares et limitées. C'est typiquement le cas des satellites, des drones (particulièrement sous-marins) ou autres dispositifs destinés à des environnements éloignés et/ou inaccessibles.

Lorsque l'on déploie un tel système, on a intérêt à s'assurer qu'il est robuste et *résilient* ; la résilience d'un système doit ici être entendue comme *la capacité de ce système à continuer d'assurer ses fonctionnalités primaires malgré les avaries qu'il connaît*. Dit autrement, il est bien qu'un satellite continue d'émettre du signal même si une de ses batteries est partie en fumée.

Pour répondre à cette problématique, les ingénieurs électroniciens conçoivent ce genre de système avec en tête des contraintes, dont on sait que la satisfaction entraîne l'absence de certaines fautes sur un système. Le problème ici est que cette approche est encore très « manuelle » : l'ingénieur réalise un design, puis observe la validité des contraintes à la main.

Nous sommes ici dans un cas typique où l'ingénierie dirigée par les modèles peut nous être d'un grand secours : les ingénieurs électroniciens ne sont pas informaticiens, mais leurs problématiques sont très facilement réglées par l'informatique. Il reste donc à faire la passerelle entre les deux, et c'est là que vous intervenez.

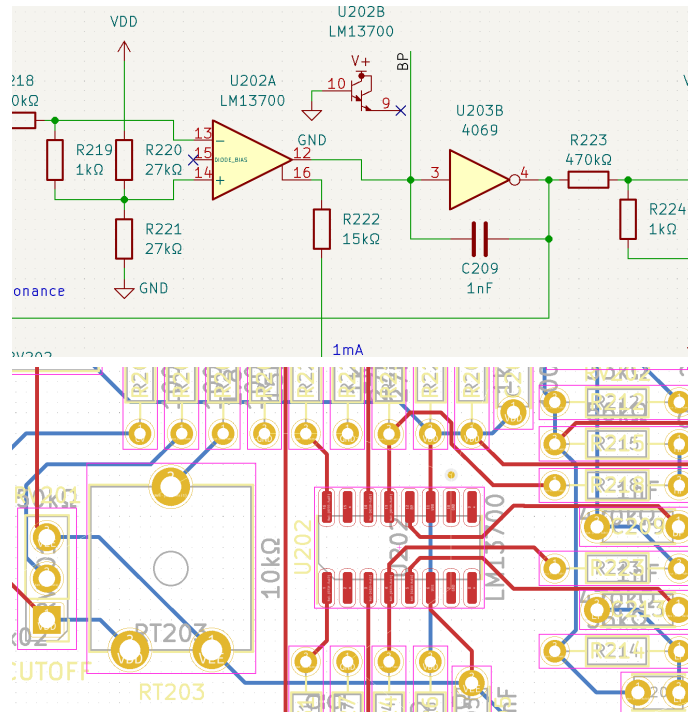


FIGURE 1 – Un logiciel de DAO pour l'électronique (KiCAD), avec le schéma (netlist) en haut et le PCB (layout) en bas – Les formes roses représentent l'encombrement des composants

1 Proposition

Le but de ce projet est de proposer un ensemble d'outils (une « suite ») pour la modélisation et la validation de circuits électroniques, basés sur la plateforme Eclipse et les technologies EMF. Dans cet optique, un utilisateur doit être capable de fournir un *catalogue* de composants associés à des contraintes, de modéliser un circuit électronique ou *netlist* (assemblage de composants connectés entre eux) et un *layout* (disposition des composants sur une *board*, reliés entre eux par des *pistes*). Une fois cela fait, l'utilisateur doit pouvoir vérifier les contraintes générales inhérentes à un design (par ex. pas de chevauchement de composants), vérifier la cohérence entre le *layout* et la *netlist*, et bien sûr de vérifier le respect des contraintes pour chaque composant.

2 Spécification partielle et travail demandé

Les objectifs pédagogiques principaux du projet sont 1) de vous confronter à des technologies d'ingénierie dirigée par les modèles dans un contexte « réaliste », et 2) d'approfondir et d'expérimenter avec les technologies abordées durant l'UE.

Un des points essentiels du projet est qu'il est réalisé de manière très autonome : c'est à vous d'apporter vos solutions personnelles au problème posé, en toute liberté ; ce projet se contentera de vous donner une *spécification* (partielle), et bien sûr quelques livrables attendus.

2.1 Fonctionnalités attendues

Nous donnons ici la liste *minimale* des fonctionnalités (ou *features*) à réaliser. Libre à vous d'en rajouter si vous en éprouvez le besoin.

F1 L'utilisateur doit pouvoir créer et consulter des *catalogues de composant*

F1.1 Un composant est défini par des *métadonnées* (nom, fabricant, etc.) que l'utilisateur doit pouvoir définir à loisir.

F1.2 Un composant doit pouvoir être associé à une *empreinte*, c'est-à-dire la place que ce dernier prend sur un circuit imprimé (par ex. un rectangle de 10 mm par 12 mm).

F1.3 Un composant définit des *ports*, qui sont ce par quoi il est connecté, et doivent apparaître sur l'empreinte.

F1.4 L'utilisateur doit pouvoir définir des contraintes sur les composants, à l'aide d'un langage de contraintes spécifique. Ces contraintes peuvent être (au minimum) :

- La combinaison de plusieurs contraintes (et, ou, non)
- Des contraintes géométrique avec d'autres composants généraux (aire à maintenir vide)
- Des contraintes géométrique avec d'autres composants spécifiques (distance à un composant avec un métadonnée particulière)
- Des contraintes en relation avec la/les *boards* du *layout*
- Des contraintes de redondance (composant doit être présent en double ou triple)
- Des contraintes sur les métadonnées du *layout*

F1.5 Le catalogue doit être accompagné d'une interface ergonomique pour sa visualisation et son édition.

F2 L'utilisateur doit pouvoir créer et consulter des *netlist*

F2.1 Nous ne vous demandons pas de réaliser un logiciel de design assisté par ordinateur, mais on doit pouvoir au moins visualiser quel composant/port est connecté à quoi.

F2.2 L'interface utilisateur doit être graphique et réalisée avec Sirius.

F2.3 Le modèle doit permettre la définition de *commentaires* sur les composants ou sur le circuit en général.

F2.4 L'utilisateur doit pouvoir exporter une *bill of materials* (BOM) depuis la *netlist*, c'est-à-dire un tableau récapitulant l'ensemble des composants du circuit avec ses méta-données d'intérêt (sélectionnées par l'utilisateur, par exemple).

F3 L'utilisateur doit pouvoir créer et consulter des *layout*

F3.1 De même, l'objectif n'est pas de faire un logiciel de design assisté par ordinateur.

F3.2 Un *layout* se compose de plusieurs *boards*, chaque *board* se composant de multiples *couches*, qui peuvent être internes ou externes (il y a au plus deux couches externes).

F3.3 Les couches externes (et *seulement* les couches externes) peuvent présenter des composants, qui sont donc associés à des coordonnées cartésiennes

F3.4 Chaque couche présente des *pistes*, autrement dit des traits de formes diverses reliant les composants entre eux.

F3.5 L'utilisateur doit pouvoir vérifier la cohérence entre le *layout* et la *netlist*, et vérifier que les contraintes associées à chaque composant sont bien respectées.

F3.6 L'utilisateur doit pouvoir exporter le *layout* vers un ensemble d'images représentant chaque couche, pour visualisation¹.

2.2 Un scénario possible d'utilisation de la suite

Plaçons-nous du côté de l'utilisateur, et voyons à quoi ressemble une utilisation typique de votre suite d'outils. Comme ébauché dans la partie précédente, la suite s'articule autour de trois activités principales : la saisie de composants dans un catalogue, la constitution de la *netlist* et la création d'un *layout*.

À noter que, dans cette section, les schémas ne sont que des ébauches présentant des représentations possibles ; libre à vous de les adapter ou de vous en défaire afin de proposer la solution que vous jugerez la plus adaptée au problème posé.

Saisie du catalogue Le catalogue regroupe tous les composants dont l'utilisateur a besoin. Chaque composant est associé à un ensemble de méta-données (nom, marque, etc.), ainsi que potentiellement à une représentation schématique et à son « emprunte », c'est-à-dire comment il se présente sur un *layout*.

L'utilisateur va donc par exemple (cf Figure 2) entrer un composant spécifique, modifier ses méta-données, faire le lien avec ses représentations, et surtout **définir des contraintes**, qui doivent être respectées lorsqu'il constituera le *layout*.

Constitution de la *netlist* L'utilisateur va fournir une *netlist* sous une forme ou une autre. Elle peut être extraite d'un autre outil, ou peut être décrite à l'aide d'un éditeur ergonomique. Dans

1. Nous recommandons le format SVG, puisqu'il repose sur un fichier texte, assez simple à générer

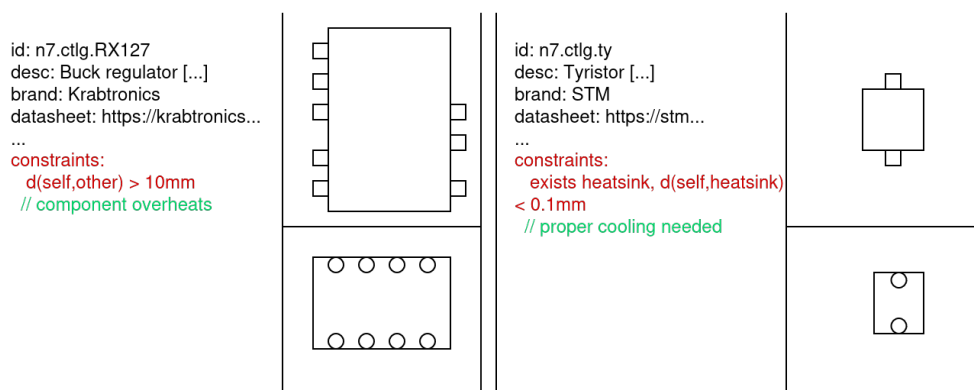


FIGURE 2 – Exemple de catalogue de composants

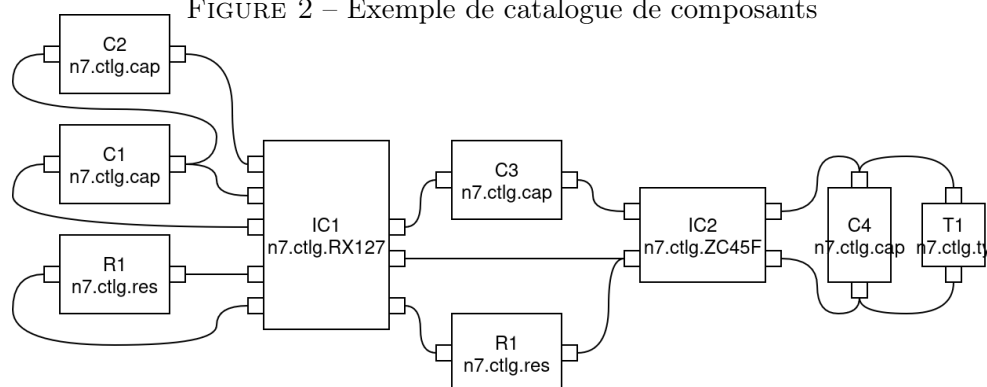


FIGURE 3 – Exemple de *netlist*

tous les cas, l'utilisateur doit pouvoir relier des composants entre eux, composants qui doivent être reliés à ceux du catalogue. La Figure 3 présente une vue possible de cette *netlist*.

L'utilisateur extrait ensuite une *BOM* à partir de la *netlist*, par exemple ici :

ID	Composant	Valeur	Matériau	Remarque	...
C1	n7.ctlg.cap	100nF	MLCC	À coller sur le circuit	...
C2	n7.ctlg.cap	100nF	MLCC	...	...
C3	n7.ctlg.cap	1000µF	Électrolytique	Pâte thermique en-dessous	...
R1	n7.ctlg.res	10Ω	Carbone	...	...
R2	n7.ctlg.res	10Ω	Carbone	...	...
IC1	n7.ctlg.RX127	N/A	N/A	Environs dégagés	...

Création du *layout* Enfin, l'utilisateur va élaborer un *layout*, c'est-à-dire disposer des composants et des pistes sur les faces d'une *board*. Une fois cela fait, il vérifie que les pistes relient les composants en suivant la *netlist*, et que les contraintes sont bien respectées. Bien sûr, si ce n'est pas le cas, il doit modifier le *layout* pour les satisfaire (cf Figure 4).

L'utilisateur a ensuite la possibilité d'exporter chaque face du *layout* sous la forme d'une image pour visualisation/revue. On peut même envisager une vue 3D du circuit !

2.3 Remarques et conseils

Imprécisions, ambiguïtés et manquements de la spécification Les zones floues dans les fonctionnalités demandées sont **volontaires**. Partout où quelque chose n'est pas précisé ou

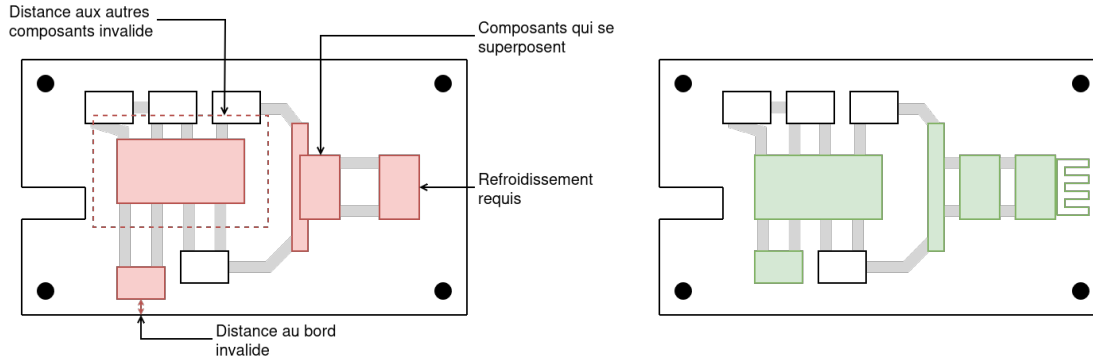


FIGURE 4 – Exemple de layout invalide (à g.) et valide (à d.)

expliqué en détail, c'est à **vous** de prendre des décisions et de faire des choix. Ces choix doivent être explicités, expliqués et argumentés, bien évidemment.

Au sujet des méta-modèles Le projet aura pour cœur *des* méta-modèles. Il est possible de tout mettre dans un seul méta-modèle mais c'est généralement une mauvaise idée (pour des raisons de séparation des préoccupations, de maintenance, de travail en groupe, etc.).

Soyez rigoureux sur la création et la maintenance des méta-modèles. Pensez notamment à leur donner des noms, URI et namespaces différents.

Au sujet des références croisées EMF permet à un méta-modèle d'importer un autre méta-modèle, et ainsi de faire référence à des objets extérieurs. Simultanément, un modèle peut charger un autre modèle, ce qui permet au éléments du modèle initial de référencer des éléments du modèle chargé. On appelle ça des *références croisées* ou *cross-references*.

On a donc deux cas de figure : 1) méta-modèle A référence méta-modèle B, modèle MA conforme à A référence modèles MB₁, MB₂, ... conformes à méta-modèle B, ce qui permet d'utiliser des concepts de B dans des modèles de A, 2) méta-modèle A, modèle MA conforme à A référence modèles MA₁, MA₂, ... conformes à méta-modèle A, ce qui permet d'avoir un modèle qui utilise un autre modèle similaire ou découper son modèle en plusieurs parties/modules réutilisables.

À noter que si le méta-modèle A utilise le méta-modèle B, il faut générer les sources du méta-modèle B *d'abord*, puis référencer le genmodel de B lors de la création du genmodel de A, afin que les sources de A contiennent les références correctes aux sources de B.

Au sujet des syntaxes concrètes On rappelle qu'il vaut mieux avoir une syntaxe abstraite claire et facile à utiliser que adaptée à une syntaxe concrète précise. En général, il est plus avantageux de faire un méta-modèle par syntaxe concrète et une transformation de modèle à modèle pour réduire les modèles issus de la syntaxe concrète à des modèles conformes à la syntaxe abstraite.

Nous vous laissons volontairement libres sur les syntaxes concrètes à utiliser. Il peut s'agir d'éditeurs arborescents si vous estimez que c'est pertinent, mais nous vous encourageons vivement à réaliser des syntaxes diverses et à essayer de vous porter sur des technologies adaptées pour chacun de vos méta-modèles. Dans tous les cas, vos choix devront être justifiés.

3 Rapport

Pour des raisons évidentes de diversité des rendus, le livrable principal du projet est un *rapport*. Ce rapport doit expliquer en détail le travail réalisé, et en particulier la *couverture fonctionnelle* (quelles fonctionnalités sont présentes et à quel point elles répondent à la spécification), les choix de conception et les compléments apportés à la spécification, le cas échéant. Il doit également présenter un regard critique sur les travaux réalisés : ergonomie, passage à l'échelle, manquements, améliorations possibles, etc.

Le rapport devra être rédigé dans une police d'écriture avec empattement (appelé aussi *serif*, par ex. Times, Garamond, etc.) et en taille 11 points. Il devra être rédigé sur du papier A4 avec des marges verticales de 3 centimètres et des marges horizontales de 2,5 centimètres.

Il devra comporter *impérativement* les éléments suivants :

- Une page de garde avec le numéro et les membres du groupe ainsi qu'un titre
- Une table des matières
- Une liste des figures
- Une introduction
- Une conclusion, qui doit inclure un bilan sur le projet (soit personnel soit de groupe), expliquant notamment les points de difficulté, et si possible une critique sur le sujet proposé
- Une description détaillée de ce que contient le rendu (cf Section 4) : description succincte de chaque projet et des fichiers importants (méta-modèles, modèles exemple, fichiers de description, scripts, etc.).

Chaque méta-modèle doit être donné sous forme d'un schéma (diagramme de classe Ecore) lisible et commenté.

Il est très vivement recommandé de faire apparaître des *figures* des outils réalisés, ainsi que des schémas pour certains aspects (les transformations ou chaînes de transformations, typiquement). Il est déconseillé de faire apparaître du code, excepté si c'est une entrée à un outil (syntaxe concrète textuelle par exemple).

L'usage d'un modèle de langage et assimilés (type GPT) pour générer tout ou partie du rapport est strictement interdit dans le cadre de ce projet.

Conformément à l'article L122-5 n° 3 du code la propriété intellectuelle, il est possible d'utiliser une oeuvre rendue publique pour contribuer à son discours à condition d'en indiquer clairement et sans ambiguïté la provenance et l'auteur, et de clairement identifier la citation en tant que tel.

4 Autres livrables

Outre les éléments demandés dans ce sujet, vous développerez un ou plusieurs exemples originaux du type de celui expliqué à la section 2.2. Ils permettront de montrer que vous avez bien compris les objectifs de ce projet. Ils seront utilisés dans la démonstration qui sera faite lors de l'oral du projet.

Le rendu se fera par **Git** obligatoirement, et se composera, en plus du rapport, de tous les workspaces Eclipse constitués dans le cadre du projet. Cela doit normalement inclure un ou plusieurs workspaces « principaux » contenant les méta-modèles, ainsi que des workspaces d'Eclipse de

déploiement contenant (par exemple) des spécifications Sirius et Xtext, des exemples de modèles, etc. On rappelle que vous devez donner une description du contenu important de ces workspaces (divers projets et fichiers que vous avez pu créer ou éditer) dans le rapport.

Vous devez accorder un soin tout particulier à la rigueur et à la propreté de votre environnement de travail. Adoptez des règles de nomenclature en accord avec les conventions Java, rangez vos workspaces de manière rationnelle, utilisez la *qualification* (`aaa.bbb.ccc...`) pour rapprocher ou séparer les éléments qui doivent l'être.

Un dépôt Git sur l'instance Gitlab de l'école vous sera fourni, mais c'est à vous de gérer ce dépôt en autonomie. Libre à vous de l'utiliser pour travailler en groupe ou de passer par un autre dépôt/autre gestionnaire de version. Notez que vous aurez les droits pour créer et fusionner des branches sur le dépôt de l'école.

Dans tous les cas, votre dépôt doit comporter une branche `main`. C'est cette branche qui servira à l'évaluation, et qui contiendra donc les workspaces, ainsi que le rapport, à la racine du dépôt.

5 Oral

L'oral a pour but de montrer ce qui a été traité du projet. Il s'agit donc de faire une démonstration des différents outils pour montrer qu'ils fonctionnent. Les détails de comment ils ont été réalisés doit être expliqué dans le rapport et n'ont pas à être abordés pendant la démonstration.

Nous vous conseillons vivement de préparer un ou plusieurs *exemples* qui serviront à illustrer les diverses fonctionnalités réalisées. Il est certainement utile de préparer un scénario à répéter, afin de ne pas hésiter pendant l'oral.

L'oral dure 20 minutes, incluant 5 minutes de questions. Un temps de préparation vous est laissé, vous permettant de mettre en place vos démonstrations. L'ordre de passage est tiré au sort, et vous sera indiqué ultérieurement.

6 Critères d'évaluations

À titre indicatif, nous donnons ici les principaux critères d'évaluation. Le poids respectif de chacun de ces aspects n'est volontairement pas explicité.

- Rapport :
 - Respect des consignes (= tous les éléments demandés sont présents, etc.)
 - Structure cohérente et pertinente (= les éléments sont organisés proprement et de manière lisible et intelligente)
 - Complétion et pertinence (= vous parlez de tout ce qui est important)
 - Figures (= vous proposez des diagrammes/listings/... lisibles et pertinentes, que vous commentez dans le discours)
 - Orthographe, grammaire, normes typographiques
- Rendu :
 - Propreté et organisation du rendu

- Noms pertinents (fichiers, projets)
- Respect des conventions (noms de projet, de paquets, de classes, d'attributs...)
- Commentaires et documentation partout où c'est nécessaire
- Description détaillée du contenu du rendu dans le rapport
- Oral :
 - Couverture exhaustive (= vous montrez tous les outils développés)
 - Pertinence (= vous ne montrez que ce qui est important)
 - Préparation (= tout est déjà en place et vous ne cherchez pas les fichiers)
 - Fluidité et maîtrise du sujet (= vous savez ce que vous faites et où vous allez)
 - Répartition de la parole (= tout le monde parle)
 - Réponses aux questions

La date de l'oral et la date butoir pour le rendu vous seront communiquées ultérieurement. Nous vous conseillons de consulter régulièrement la page moodle de l'UE pour vous tenir à jour sur les détails relatifs au projet.